

Функции и процедуры. Рекурсия

часть 2

Цель

Научиться создавать блоки кода, которые могут быть многократно использованы в программе для ее структурирования, эффективности и ускорения процесса разработки

Что нужно сделать

Решите следующие задачи на языке программирования Python

01 Вычисление степени без цикла

Вам необходимо написать функцию `power`, вычисляющую значение N в степени M без использования цикла и оператора возведения в степень с помощью рекурсии

Формат ввода

Аргументы функции - два целых числа N и M (N - положительное, M - неотрицательное)

Формат вывода

Требуется вернуть значение N в степени M

Необходимые данные для решения и проверки

Дано	Вывод данных
<code>power(2, 3)</code>	8
<code>power(2, 0)</code>	1
<code>power(5, 3)</code>	125

02 Палиндром без цикла

Вам необходимо написать функцию `is_palindrome`, определяющую, является ли список палиндромом. (Палиндром - строка или список, которая читается слева направо и справа налево одинаково).

Использовать цикл, создавать другой список и изменять текущий нельзя

Формат ввода

Аргумент функции - список из целых чисел

Формат вывода

Требуется True, если список является палиндромом, и False в ином случае

Необходимые данные для решения и проверки

Дано	Вывод данных
<code>is_palindrome([2, 3, 4, 3, 2])</code>	True
<code>is_palindrome([2, 3, 3, 2])</code>	True
<code>is_palindrome([2])</code>	True
<code>is_palindrome([2, 3, 4, 3, 1])</code>	False

03 Арифметическая прогрессия

Вам необходимо написать функцию, которая возвращает другую функцию, которая при каждом вызове возвращает элемент арифметической прогрессии. Использовать замыкание. Прогрессия строится по первому элементу и шагу

Формат ввода

Аргументы функции - start и step - первый элемент и шаг соответственно

Формат вывода

Функция, которая при каждом следующем вызове будет возвращать новый элемент прогрессии

Необходимые данные для решения и проверки

Дано	Вывод данных
<code>f = progression(3, 5)</code> <code>f()</code> <code>f()</code> <code>f()</code>	3 8 13
<code>f = progression(12, -7)</code> <code>f()</code> <code>f()</code> <code>f()</code> <code>f()</code>	12 5 -2 -9

Дано	Вывод данных
<pre>f = progression(6, 0) f() f() f()</pre>	<pre>6 6 6</pre>

04 Аргументирующий декоратор

Вам необходимо написать декоратор `@arg_decor`, который перед каждым вызовом обернутой в него функции печатает значение аргумента и его тип

Пример

```

ooo

@arg_decor
def some_func(a, b, c):
    print(f'{a}.{b}.{c}')

>>> some_func(3, 'Помидор', 3.14)
>>> value=3 type=int
>>> value=Помидор type=str
>>> value=3.14 type=float

```

Формат ввода

-

Формат вывода

Перед каждым вызовом обернутой в него функции печатает значение аргумента и его тип через пробел с новой строки

Необходимые данные для решения и проверки

Дано	Вывод данных
<pre>@arg_decor def some_func_1(a, b, c): print(f'{a}.{b}.{c}') some_func(3, 'Помидор', 3.14)</pre>	<pre>value=3 type=int value=Помидор type=str value=3.14 type=float 3.Помидор. 3.14</pre>
<pre>@arg_decor def some_func_2(*cool_args): pass some_func_2(3, 'Помидор', 3.14)</pre>	<pre>value=3 type=int value=Помидор type=str value=3.14 type=float</pre>

05 Скобочные последовательности

Вам необходимо написать функцию `bracket_sequences`, которая возвращает все варианты **правильных скобочных последовательностей** для одного вида скобок (круглых), принимающая в качестве аргумента число парных скобок

Формат ввода

Аргумент функции - число пар скобок (неотрицательное целое)

Формат вывода

Функция возвращает список неповторяющихся строк, которые образуют множество всех возможных правильных скобочных последовательностей с заданным количеством пар скобок (список отсортирован в лексикографическом порядке)

Необходимые данные для решения и проверки

Дано	Вывод данных
<code>bracket_sequences(0)</code>	<code>[]</code>
<code>bracket_sequences(1)</code>	<code>['()']</code>
<code>bracket_sequences(2)</code>	<code>['(())', '()()']</code>

Результат

В ответе приложите файл с расширением `ru` для каждой задачи

Критерии оценивания

K1	Решено верно 2 задачи, пройдены все тесты	2 балла
-----------	---	----------------

K2	Решено верно 3 задачи, пройдены все тесты	3 балла
-----------	---	----------------

K3	Решено верно 4 задачи, допускается непрохождение 1 теста из всех	4 балла
-----------	---	----------------

K4	Решено верно 5 задач, допускается непрохождение до 2 тестов из всех	5 баллов
-----------	--	-----------------

Максимальное количество баллов	5 баллов
---------------------------------------	-----------------

Минимальное количество баллов чтобы преподаватель смог зачесть вашу работу	2 балла
--	----------------